Using open-source code for gas turbine performance modelling

Tutorial of Basics GT2026-TOB-01-03

ASME IGTI TE 2026 **Aircraft Engine Committee**

Dr. Wilfried Visser



Oscar Kogenhop MSc



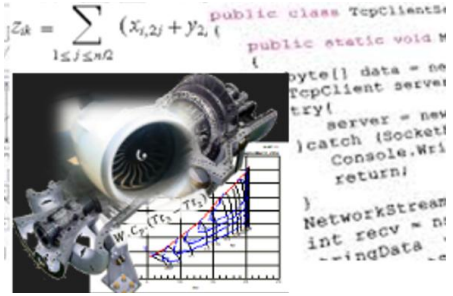
Overview

- Introduction
- Propulsion and power system modelling
- GSPy (version 2.0 !)
  - Python software implementation
  - Example case: jet engine performance model
  - Demonstration
  - New features version 3.0 (beta)
- Concluding remarks

For whom is this tutorial?

- Engineers, scientists, researchers, students
  - With interest in system simulation of propulsion and power systems
  - Wanting to learn how system models work
  - Needing models with new functionalities
    - Coupling with other models
  - Having basic coding skills





Key focus areas

- System modelling

today: gas turbines

- Object orientation
- Python programming language
  - Open-source libraries
    - Aero-thermal
    - Numerical methods
    - Data handling
    - Visualization

```
1  class CodeExample():
2      def printStatement(self):
3          print('Hello World!')
4
5  def main():
6      classEx = CodeExample()
7      classEx.printStatement()
8  if __name__ == "__main__":
9      main()
```

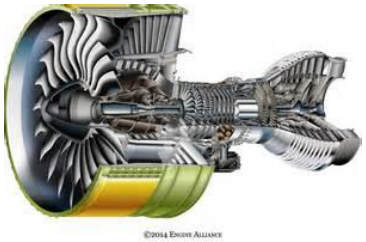
System performance model

- System of interacting components
  - Component models (usually 0-D, 1-D …)
  - Relationships among components
- Equations
  - conservation of mass, energy, momentum, other…
  - mostly non-linear algebraic/differential
- Component operating points
  - Defined by state variables
- Solution of set of equations
  - State variable vector representing valid system operating point
  - Steady-state or transient

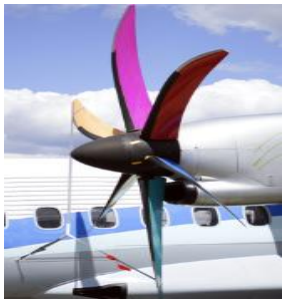
Propulsion and power systems

System performance modeling

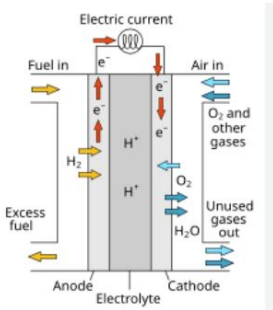
- Gas turbines
  - Aero engines: jet engines, turboprop, turboshaft
  - Land based engines: turboshaft
- Propellers and rotors
- Hybrid elements
  - Electric motors
  - Generators
  - Fuel cells
  - Batteries
- Alternative/sustainable fuels











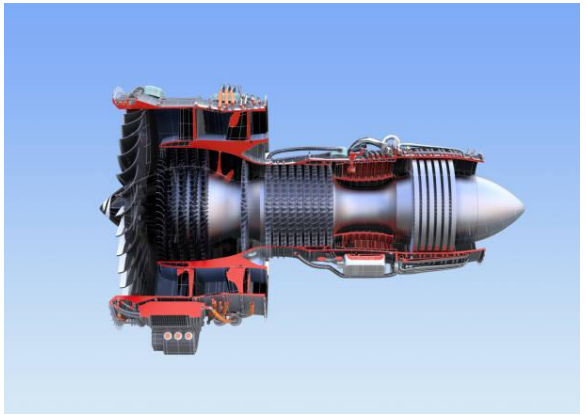


Propulsion and power system performance prediction

System performance modeling

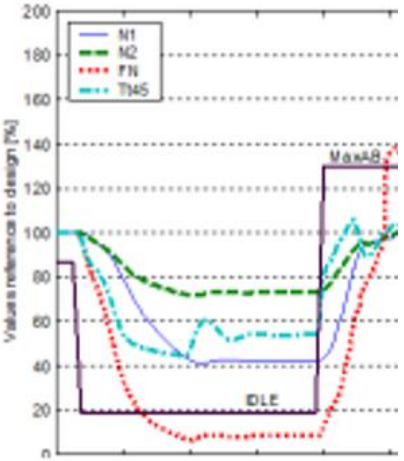
- Design studies





- Existing designs
  - off-design performance

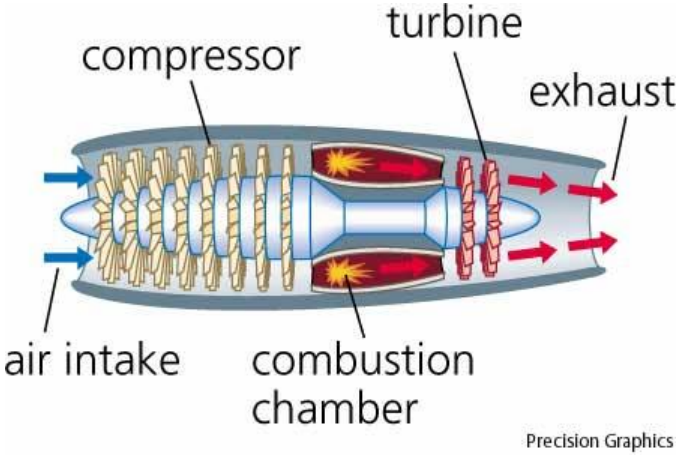




Gas turbine system performance models

System performance modeling

- Component models
  - intake, fans/compressors, combustors, turbines, mixers, exhausts,
  - Component maps
- State variables
  - Rotor speeds, mass flows, pressure ratios, …



- Equations
  - Mass flow in = mass flow out (at component intersections, engine stations)
  - Power balance : PWcompressor = PWturbine
- Input
  - Fuel flow, TIT, inlet P and T…
- Output
  - Thrust FN, Power, Fuel flow, Efficiency, P, T at engine stations…

Design point simulation

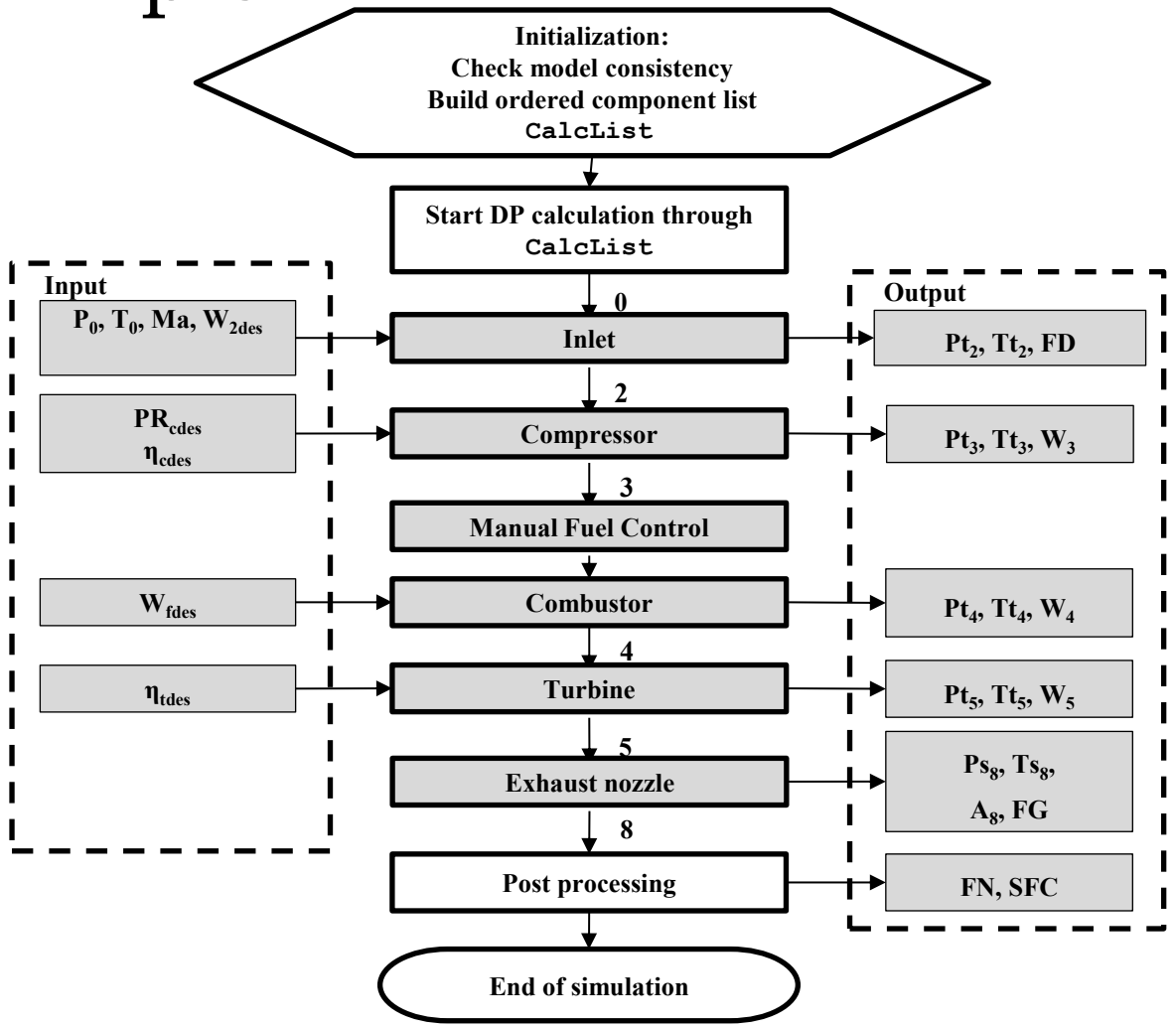
System performance modeling

- Design studies
- Effects of cycle design parameters *(on performance req's)*
  - e.g. PR, BPR, W, efficiencies
- Varying dimensions
- Simple
  - straight forward inlet to exhaust calculation
  - conservation of mass, energy, momentum
- More complex
  - solving equations
  - Constraints
  - AI methods for optimization

Design point calculation example

- Single spool Turbojet
- Simple:
  - No iteration

Design point simulation



Off-design ('OD') simulation

System performance modeling

- Analyze effects of operating conditions on performance
- Fixed geometry and dimensions

**except for variable geometry features like VSVs, IGVs, Bleed valves*

- Requires initially calculated Design point
  - 'cycle reference point' CRP
- Predict how the operating point changes (from CRP) with operating conditions deviating from the design conditions
  - Steady-state equilibrium
  - Transient (change in time, e.g. accel/decel of rpm's, heat transfer, …)

Equations and numerical methods

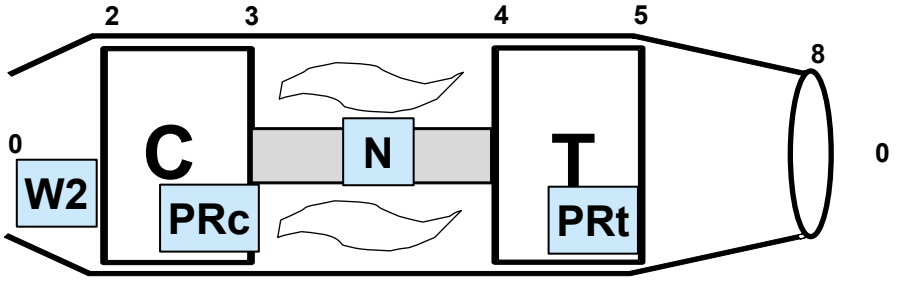
Off-design simulation

- Solve equations for a turbine engine steady-state or transient OD operating point
- State variables
  - representing the operating point
  - minimal number of independent (or 'tearing') variables
  - other parameters can be directly/explicitly calculated from the state variable values
- Find the OD operating point = find state variable values
  - Given: fuel flow and inlet conditions
  - Satisfying all conservation equations
  - Residuals ('errors') must all be 0
- Single solution
  - n 'error equations' with n unknown 'state variables'
  - Calculate all dependent other parameter values

Turbojet example - states

Off-design simulation

- Required: 4 states S1..4
  - S1 = W2
  - S2 = PRc*
  - S3 = N
  - S4 = PRt



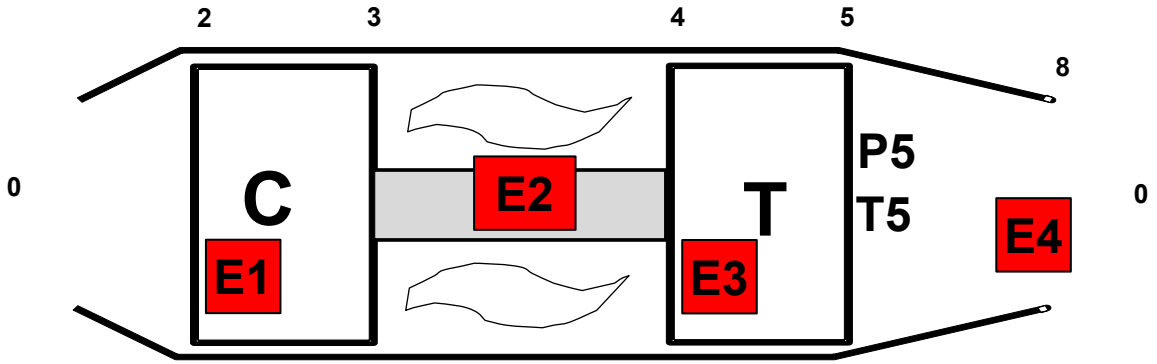
- From the 4 states all other parameters can be calculated

**Instead of PR, usually Beta is used*

Turbojet example - equations

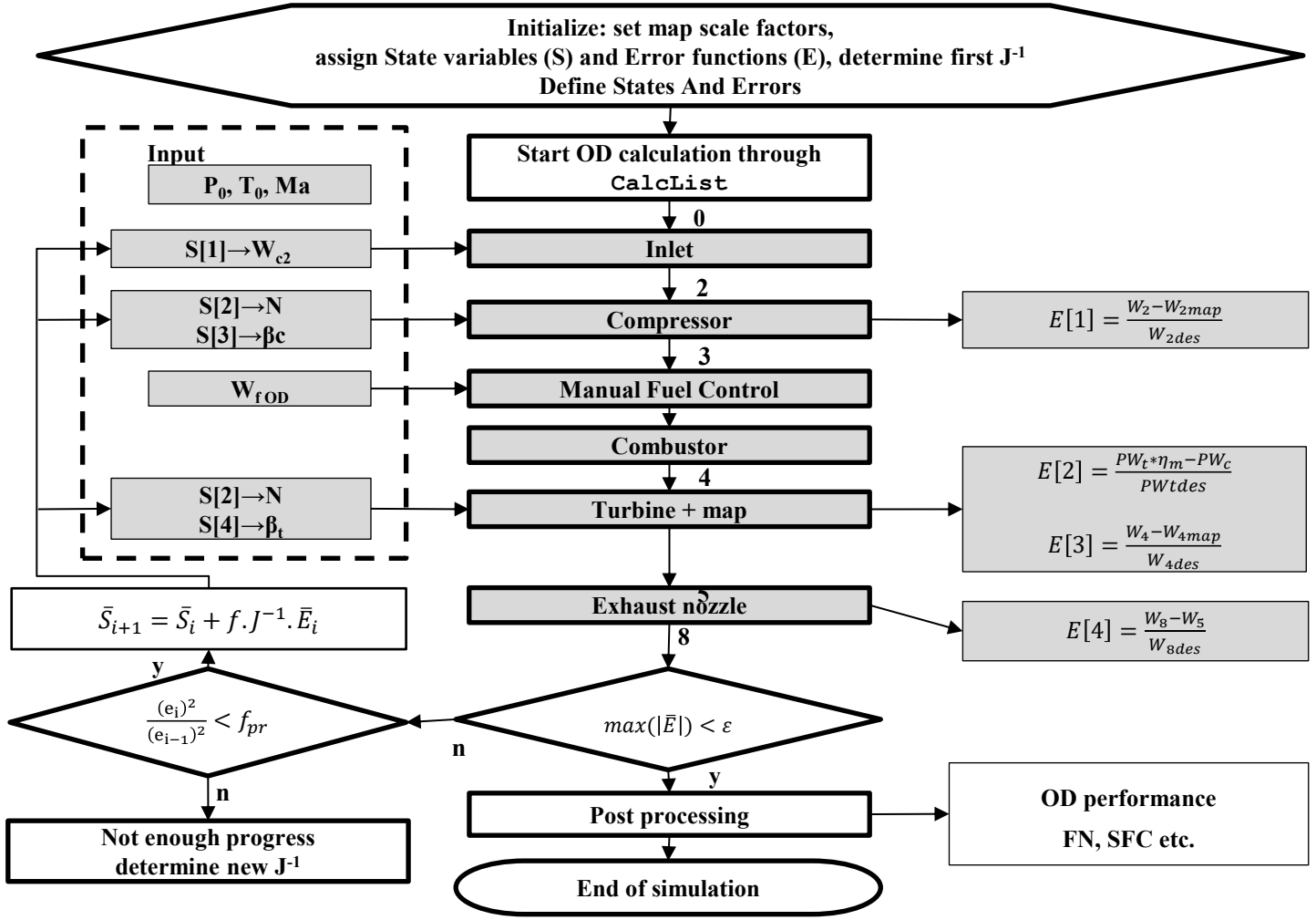
Off-design simulation

- Satisfying conservation equations
- To find the 4 unknown states we need 4 equations
  - E1 = Inlet mass flow - compressor mass flow (map) = 0
  - E2 = Compressor power + turbine power = 0
  - E3 = Combustor exit mass flow - turbine mass flow (map) = 0
  - E4 = Turbine mass flow - nozzle mass flow = 0



Off-design iteration flow chart - turbojet

Off design simulation



Recommended literature

- **AE4263 "Modelling and Simulation of Propulsion and Power Systems**
- W. Visser "Generic Analysis Methods for Gas Turbine Engine Performance: the development of the gas turbine simulation program GSP", 6 January 2015, Delft University of Technology, ISBN 978-94-6259-492-0 (available from https://repository.tudelft.nl/islandora/object/uuid%3Af95da308-e7ef-47de-abf2-aedbfa30cf63).
- H. I. H. Saravanamuttoo, G. F. C. Rogers and H. Cohen, *Gas Turbine Theory*, 5th ed.: Pearson Education, 2001.
- Several authors and contributors, 'AVT-018 Performance Prediction and Simulation of Gas Turbine Engine Operation', NATO Research and Technology Organization 2003 RTO-TR-044
- Walsh P.P, Fletcher P., Gas turbine Performance,
  ISBN-10: 063206434X ISBN-13 : 978-0632064342
- Visser, W.P.J. and Broomhead M.J., 2000, 'GSP, a generic object-oriented gas turbine simulation environment', ASME paper 2000-GT-2, presented at the ASME TURBO EXPO 2000, 8-11 May 2000, Munich, Germany,
- J. Kurzke, "Advanced User-friendly Gas Turbine Performance Calculations on a Personal Computer", 95-GT-147, presented at the ASME IGTI Turbo Expo, 1995.`
- J. Kurzke, "How to get Component Maps for Aircraft Gas Turbine Performance Calculations", ASME Paper 96-GT-164, 1996.
- W. P. J. Visser and I. D. Dountchev, "Modeling Thermal Effects on Performance of small Gas Turbines", GT2015-42744, to be presented at the ASME IGTI Turbo Expo 2015, Montreal, Canada, 2015.

New system model elements

System performance modeling

- Sustainable fuels
- Electric systems
- Heat exchangers
- Hybrid systems (gas turbines, fuel cells, batteries)
- Generators, Electric motors driving fans and propellers

- There is a need for
  - Extensions in modeling and simulation tools



Or, build tools from scratch ?

From scratch

System performance modeling



- Things to think about
  - ❖Will others use my code?
  - ❖Will code be used later?
    - ❖For other propulsion systems / engines?
    - ❖Flexibility
  - ❖Costs / Benefits of making the tool generic?
    - ❖Add a user-friendly GUI?
    - ❖Or just run from the (Python) interpreter
  - ❖?...
- Preparing your project
  - Select language, development platform
  - Design software architecture

Why from scratch?

- Pros
  - Full control and flexibility
  - "I know what I am doing"
  - Coupling with other models (1..4 D models)

- Cons
  - A lot of work
    - Validation
    - Code maintenance / documentation
  - Code accessibility / readability to others
    - Especially 'if you leave the company'

Software implementation

Do you really want to start from scratch?



Language / development environment

Software implementation

- Python
  - Open-source programming language
  - Ready to use libraries for many engineering problems
  - Widely used by students, young engineers and scientists
  - Readable code and simple syntax
  - Supports Object Orientation (OO)
  - Cross-Platform Compatibility (Windows, Linux, IOS)
  - Con's?
    - memory usage/performance limitations/library versions
- Development environments (IDE)
  - Microsoft Visual Studio Code ('VS Code'), PyCharm

Python versus previous experience

Software implementation

- Gas turbine Simulation Program GSP
  - Developed at
    - Netherlands Aerospace Centre NLR
    - Delft University of Technology
  - Object oriented, GUI
  - 4GL development environment, 30 years of legacy code
  - ~120,000 lines of code (excluding library code)

- Python "GSPy"
  - using open-source libraries

< 4000 lines of code !!!!

Working with Python models

Software implementation

- Run code from development environment (no GUI)
- Specify input in code
  - Example: specify compressor component model and design performance

```
compressor1 = TCompressor(turbojet,          # owning system model object
                          'Compressor1',     # component name
                          'compmap.map' ,    # map file name
                          None,              # optional control component
                          2, 3,              # station nr in and out
                          1,                 # shaft id, strings are allowed
                          16540,             # design rpm
                          0.825,             # design efficiency
                          1,                 # map design Nc (for scaling)
                          0.75,              # map design Beta (for scaling)
                          6.92,              # design pressure ratio
                          'GG'               # speed option
```

- Adapt / extend code where needed
- Alternatives: separate input text files, or GUI (lot of extra work)

Python for system modeling

Software implementation

- Object Oriented architecture
- Main program/model script
  - defining the model, component configurations etc.
- Using open-source libraries
- Component model implementations
  - separate source code files
- Output of simulation results

Object Orientation (OO)

Software implementation

- Encapsulation
  - An object holds both data and 'methods' (procedures, functions) working with these and data, and also external data
  - Object class (class is 'type' of object)
- Inheritance
  - An object can inherit data and methods from a 'parent class'
- Polymorphism
  - An object variable can represent objects from different classes inheriting from the same parent
- Abstraction
  - Representing the essential features without concerning about the background details

Why object orientation?

Software implementation

- Essential to build a flexible modular framework
- Rapidly implement system models with different configurations using same building blocks ('objects')
- Maintain overview ('encapsulation')
- Avoid code duplication
- Enhance code maintainability

Why object orientation?

Software implementation

- Example implementing system models with different configurations

1. Build a turbojet model
   - Inlet-compressor-combustor-turbine-exhaust nozzle

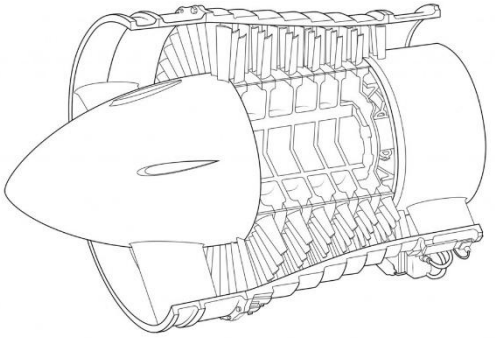
2. Build a turbofan model
   - Use the same elements as used in the turbojet model, add others
   - Without moving large chunks of code around..

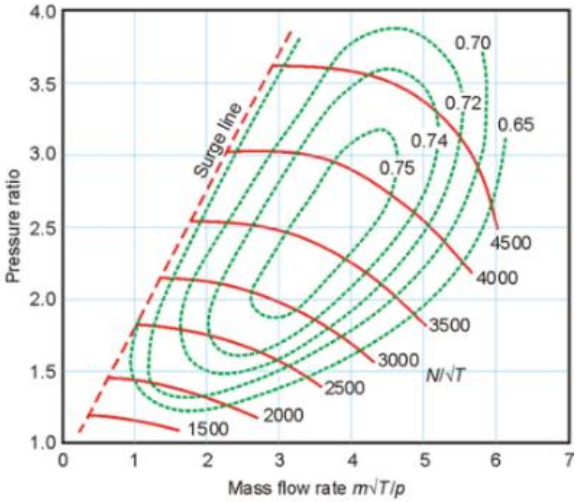
… applying inheritance, polymorphism and abstraction !

Encapsulation example

Object orientation

- Compressor component model
  - compressor object class holds
    - W, Beta (PR), Eta, N
    - functions to calculate the compression process
    - Compressor performance map
    - Functions to communicate with the system model and other component models
    - Calculate equation *residuals from states*



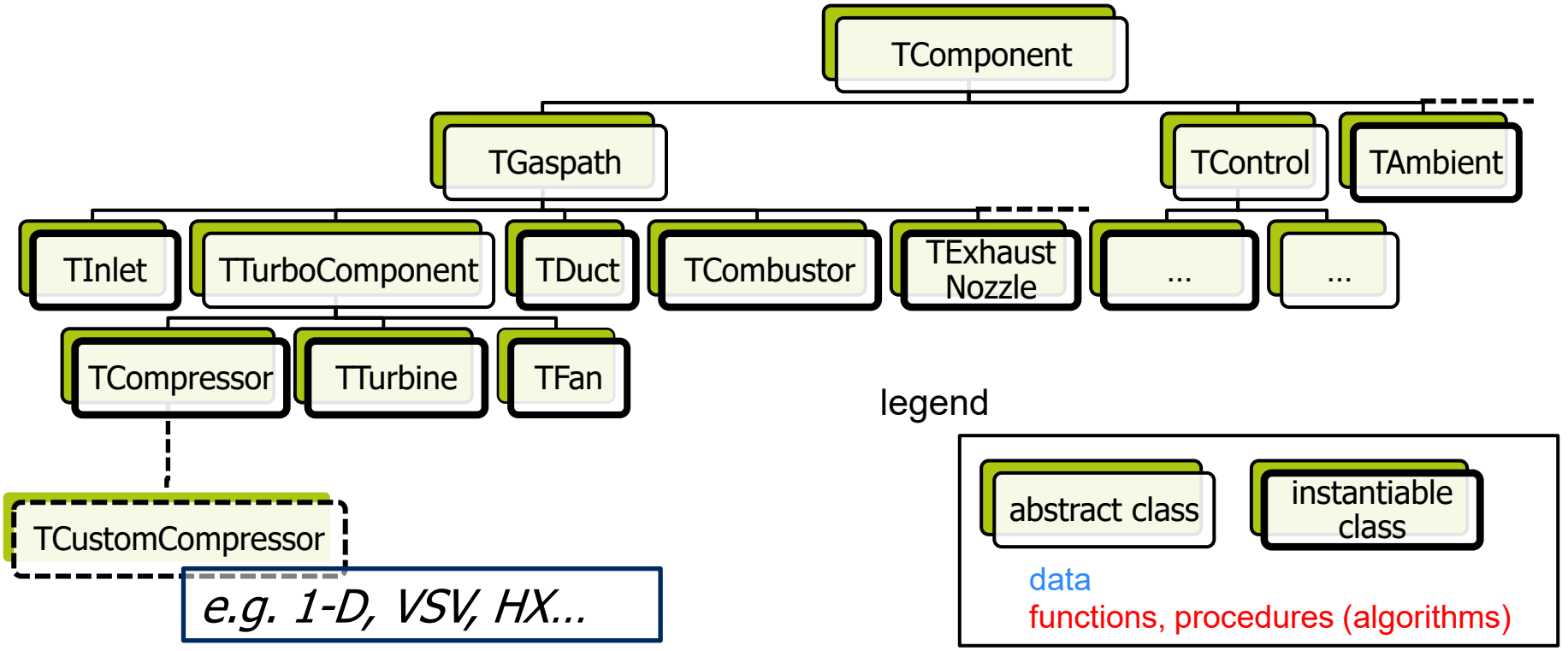


Polymorphism/abstraction

Object orientation

- Generic component model class object
  - *'abstract'* object that can represent any child object
  - has a generic 'Run' method (procedure or function)
  - Child classes inherit and 'override' the Run procedure with calculations simulating the actual process in the component
- System model
  - List of objects (of instantiable classes)
    - E.g. Inlet-compressor-combustor-turbine-exhaust
    - System model simulation subsequently calls the 'Run' methods of all objects in the list, in this example calculating from inlet through exhaust….

GSPy inheritance tree *(partial)*



2.0 model definition & polymorphism

Software implementation

```
#  create system model object:
turbojet = TSystemModel('Turbojet', model_file = __file__)

#  create component model objects, make turbojet owner:
fuelcontrol = TControl(turbojet, 'Fcontrol', '', ···
inlet1    = TInlet(turbojet, ···
compressor1 = TCompressor(turbojet, ···
combustor1 = TCombustor(turbojet, ···
turbine1 =    TTurbine(turbojet, ···
duct1     = TDuct(       turbojet,  ···
exhaustnozzle = TExhaustNozzle(turbojet, ···

# define the run list order for one cycle calc pass during iteration
turbojet.define_comp_run_list(
    fuelcontrol,
    inlet1,
    compressor1,
    combustor1,
    turbine1,
    duct1,
    exhaustnozzle)
```

one cycle calculation pass :

… going through all ***virtual*** component Run methods

```
for comp in self.component_run_list:
    comp.Run(mode, point_time)
```

Python open-source libraries

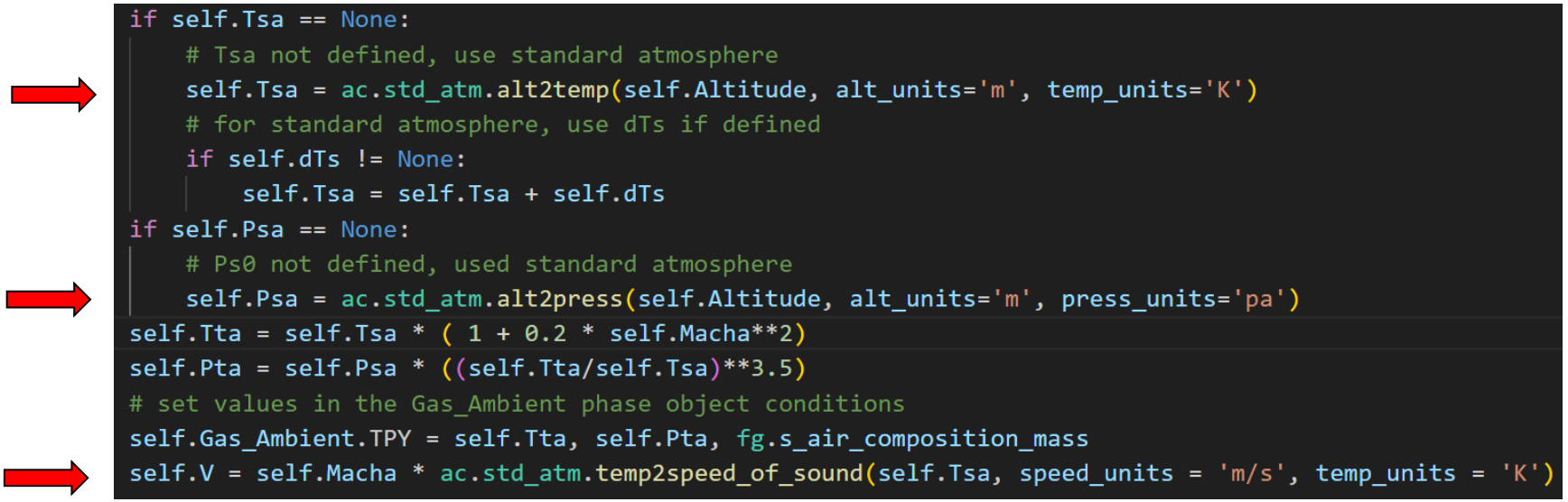
Software implementation

- Standard atmosphere — *aero-calc*
- Thermodynamics — *cantera*
  - Compression, expansion in compressors, turbine, exhausts, nozzles
  - Combustion chemical reactor model
- Multidimensional arrays — *numpy*
- Numerical methods — *scipy*
  - 'root' generic solver ('Krylov' method)
- Output data export
  - Export to .csv — *pandas*
    - Make graphs in Excel
  - Map graphs with operating curves — *matplotlib*

Aero-calc library

Software implementation

- Aero-calc functions used for calculation of ambient/flight conditions from altitude, flight Mach, dT
  - alt2temp
  - alt2press
  - temp2speed_of_sound



```
if self.Tsa == None:
    # Tsa not defined, use standard atmosphere
    self.Tsa = ac.std_atm.alt2temp(self.Altitude, alt_units='m', temp_units='K')
    # for standard atmosphere, use dTs if defined
    if self.dTs != None:
        self.Tsa = self.Tsa + self.dTs
if self.Psa == None:
    # Ps0 not defined, used standard atmosphere
    self.Psa = ac.std_atm.alt2press(self.Altitude, alt_units='m', press_units='pa')
self.Tta = self.Tsa * ( 1 + 0.2 * self.Macha**2)
self.Pta = self.Psa * ((self.Tta/self.Tsa)**3.5)
# set values in the Gas_Ambient phase object conditions
self.Gas_Ambient.TPY = self.Tta, self.Pta, fg.s_air_composition_mass
self.V = self.Macha * ac.std_atm.temp2speed_of_sound(self.Tsa, speed_units = 'm/s', temp_units = 'K')
```

Cantera library

Software implementation

- Open-source suite of tools for problems involving chemical kinetics, thermodynamics, and transport processes
- https://cantera.org/index.html
- Jetsurf mechanism used for
  - Gas path transport
  - Compression/expansion
  - Combustion

https://web.stanford.edu/group/haiwanglab/JetSurF/JetSurF2.0/Index.html

Using Cantera (ct) **gas model**

Software implementation

- define gas model

```
gas = ct.Solution('jetsurf.yaml')
```

- Set T, P, composition (mass mixing ratios)

```
gas.TPY = 1200, 101325, 'CH4:1, O2:2, N2:7.52'
```

- Or set P and S like gas.PSY = ……..

- Get T, H, S or P

```
P = gas.P      S = gas.S

T = gas.T      H = gas.H
```

Using Cantera (ct) **mass flow/transport**

Software implementation

- Define a quantity (mass flow) of gas

gas_in = ct.Quantity(gas, mass_flow)

- Set entropy or enthalpy and pressure using

gas_in.SP = entropy value, pressure value

gas_in.HP = enthalpy value, pressure value

- Isentropic compression of gas_in mass flow (with pressure ratio PR)

```
Sin = gas_in.s
Pout = gas_in.P*PR
gas_out.SP = Sin, Pout # get gas_out at constant s and higher P
his_out = gas_out.enthalpy_mass # isentropic exit specific enthalpy
```

Using Cantera (ct)

combustion

Software implementation

- chemical equilibrium reactor model
- 2 options for fuel specification

User specified composition (single species)

T and P

or

User specified LHV (mix of 'unknown' species, e.g. Kerosine, Jet-A)

T and P

average H/C and O/C ratio of the fuel molecules

- In both cases end-combustion H, T, P and composition can be calculated

User specified fuel composition

Cantera - combustion

fuel = ct.Quantity(fg.gas) *define fuel*

fuel.mass = fuel mass flow

fuel.TPY = T, P, fuel_composition *define T, P, composition*

fuel_composition is a string variable, e.g.:
'C12H26:1' *or a 5 to 1 mixture :* ***'C12H26:5, C8H12:1'***

gas_out = gas_in + fuel *mix fuel with air*

gas_out.equilibrate('HP') *chemical equilibrium combustion end state*

T4 = gas_out.T *get combustion end T (or H, S*

etc)

User specified LHV

Cantera - combustion

- More complicated
- Cannot use Equilibrate function
- Need to calculate chem. equilibrium end composition from
  - Air composition
  - Fuel H/C and O/C ratios
  - Mole fractions book keeping (converting between mass – moles)

w_fuel * LHV * 1000 + w_air * H_in = (w_air + w_fuel) * H_out

Numpy and Scipy

Software implementation

- https://docs.scipy.org/doc/
- Numpy
  - multidimensional array operations
- Scipy
  - solving system model non-linear algebraic/differential equations
  - using **Root** function
    - finding root of a vector function
      > i.e. error vector as function of the state vector
      > result is a valid system steady-state or transient point
    - newton-Raphson like iteration
    - several solver methods
  - Most suitable solver method 'Krylov'

Solver code

Software implementation

Do_Run

- runs all comp. model code as function of states
- returns vector of equation errors

```
def Do_Run(self, mode, point_time, states_par):
    # global system_model, states, errors, Ambient, Control
    self.states = states_par.copy()
    ...
    for shaft in self.shaft_list: shaft.Run(mode, point_time)
    # Run simulation code of all components in the system model
    for comp in self.component_run_list:
        comp.Run(mode, point_time)
        # load comp data into the output_dict
        self.output_dict.update(comp.get_outputs())
    ...
    return self.errors
```

- ***residuals*** is vector function (returns all 0's if states is valid OP)

```
def residuals(states):
    # Will return residuals of system conservation equations, schedules,
    # limiters etc. The residuals are the errors returned by Do_Run
    return self.Do_Run('OD', point_time, states)
```

- ***self.states*** = the initial value of the state vector

```
solution = root(residuals, self.states, method = 'krylov', options={...
```

- ***solution*** = *the state vector representing a valid operating point*

Basic script elements for a turbojet simulation

Software implementation

(key code lines only)

Create model

Component models …

Define run list order

DP simulation

OD simulation

```
#  create system model object:
turbojet = TSystemModel('Turbojet', model_file = __file__)
#  create component model objects, make turbojet owner:
fuelcontrol = TControl(turbojet, 'Fcontrol', '', …
inlet1    = TInlet(turbojet, …
#  etc ... …
# define the run list order for one cycle calc pass during iteration
turbojet.define_comp_run_list( …
# ....
# set DP ambient/flight conditions
turbojet.ambient.SetConditions('DP', 0, 0, 0, None, None)
turbojet.Run_DP_simulation()
# ...
# Run OD simulation at Alt 11 km / Ma 0.8 for series of fuel flows
turbojet.input_points = fuelcontrol.get_OD_input_points()
turbojet.ambient.SetConditions('OD', 11000, 0.8, 0, None, None)
turbojet.Run_OD_simulation()
```

Passing gas conditions in Python

Software implementation

- `comp.Run(` is run in order given in `TSystemModel` `component_run_list`

- `component_run_list` must be ordered such that
  - component gas path upstream conditions
  - other component/control parameters needed for "Run"

  have been calculated

- Basic TGaspath "Run" code takes inlet conditions from system model ***gaspath_conditions[...] dictionary***

Abstract TGaspath class

ancestor class of all gas path components

```
def Run(self, Mode, PointTime):
    self.gas_in = self.owner.gaspath_conditions[self.station_in]

    if Mode == 'DP':
        # create gas_inDes, gas_out cantera Quantity (gas_in already created)
        self.gas_inDes = ct.Quantity(self.gas_in.phase, mass = self.gas_in.mass)
        self.gas_out = ct.Quantity(self.gas_in.phase, mass = self.gas_in.mass)
        self.Wdes = fu.scalar(self.gas_inDes.mass)
        self.Wcdes = self.Wdes * fu.GetFlowCorrectionFactor(self.gas_inDes)
        self.W = self.Wdes
        self.Wc = self.Wcdes
    else:
        self.W = fu.scalar(self.gas_in.mass)
        self.Wc = self.W * fu.GetFlowCorrectionFactor(self.gas_in)
        self.gas_out.TPY = self.gas_in.TPY
        self.gas_out.mass = self.gas_in.mass

    self.owner.gaspath_conditions[self.station_out] = self.gas_out
```

Component model implementations

Software implementation

- Implementation of component performance maps
- Inlets
- Turbo machinery
  - Compressors
  - Fans
  - Turbines
- Combustors
- Ducts
- Exhaust nozzles and diffusers

TInlet run code

- TInlet is descending from **TGaspath class**
- Overriding TGaspath component ***Run*** code

```
...
self.gas_out.TP = self.gas_in.T, self.gas_in.P * self.PR
...
```

- PR < 1 for simple inlet total pressure loss

States and errors in component models

Software implementation

- The system model requires an equal number of
  - state variables ('states')
  - and error functions (equation residuals)
- Component model code
  - initializes states and/or errors
  - depending on the type of component
  - See turbojet OD turbojet simulation example slides
    - 4 states - 4 errors
- Component map input parameters typically become states:
  - Nc and Beta for a compressor

TInlet code - adding a state variable

Component model implementation

```
def Run(self, Mode, PointTime):
    if Mode == 'DP':
        # Get the ambient conditions for the inlet gas_in conditions
        self.owner.gaspath_conditions[self.station_in] = self.owner.gaspath_condi
        self.owner.gaspath_conditions[self.station_in].mass = self.Wdes
    super().Run(Mode, PointTime)
    # self.gas_in.TP = self.gas_in.T, self.gas_in.P
    if Mode == 'DP':
        self.gas_in.mass = self.Wdes
        self.wcdes = self.gas_in.mass * fu.GetFlowCorrectionFactor(self.gas_in)
        self.wc = self.wcdes
        self.PR = self.PRdes
        self.owner.states = np.append(self.owner.states, 1)
        self.istate_wc = self.owner.states.size-1   # add state for corrected inl
    else:
        self.wc = self.owner.states[self.istate_wc] * self.wcdes
        ...
```

Adding state variable

TCompressor - adding 2 states & 1 error

Software implementation

```
def Run(self, Mode, PointTime):
    super().Run(Mode, PointTime)
    if Mode == 'DP':
        self.PW = fu.Compression(self.gas_in, self.gas_out, self.PRdes, self.Etades,
        self.ReadTurboMapAndSetScaling()
        # add states and errors
        if self.SpeedOption != 'CS':
            if self.shaft.istate == None:
                self.owner.states = np.append(self.owner.states, 1)
                self.istate_n = self.owner.states.size-1
                self.shaft.istate = self.istate_n
            else:
                # already assigned (e.g. by fan or compressor upstream in the gas pat
                self.istate_n = self.shaft.istate
        self.owner.states = np.append(self.owner.states, 1)
        self.istate_beta = self.owner.states.size-1
        # error for equation gas_in.wc = wcmap
        self.owner.errors = np.append(self.owner.errors, 0)
        self.ierror_wc = self.owner.errors.size-1
        # calculate parameters for output
        self.PR = self.PRdes
    else:
```

Adding state variables

note that the N state is held by shaft object

Adding error variable

Compressor - Off-design mode

Component model implementation

States and errors at work….

TShaft
Run sets N from state variable

```
def Run(self, Mode, PointTime):
    if Mode == 'DP':
        self.Nt = self.Ntdes
    else:
        self.Nt = self.owner.states[self.istate] * self.Ntdes
```

TCompressor
N from shaft

Beta from state variable

error calculation

```
else:  # i.e. OD simulation mode
    if self.SpeedOption != 'CS':
        self.N = self.shaft.Nt
    self.Nc = self.N / fu.GetRotorspeedCorrectionFactor(self.gas_in)
    self.Wc,self.PR,self.Eta=self.GetTurboMapPerformance(self.vg_angle, self.Nc,
                                                          self.owner.states[self.istate_beta])
    self.PW = fu.Compression(self.gas_in, self.gas_out, self.PR, self.Eta, self.Polytropic_Eta)
    self.W = self.Wc / fu.GetFlowCorrectionFactor(self.gas_in)
    self.owner.errors[self.ierror_wc ] = (self.W - self.gas_in.mass) / self.Wdes
    self.gas_out.mass = self.W
```

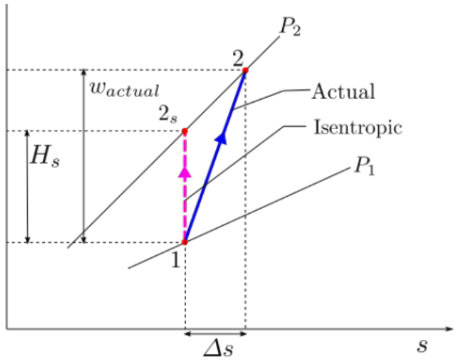
Implementation of other components

- Generic functions
  - Compression
  - Expansion
- See code for
  - Combustion (also see slides on Cantera)
  - Duct: simple pressure loss
  - Exhaust nozzle: isentropic expansion
  - Control
  - Ambient conditions (see slides on Aerocalc)

Compression code

Component model implementation

- Generic function used for calculating compressor or fan end conditions
  - Uses Cantera
  - Eta isentropic or polytropic

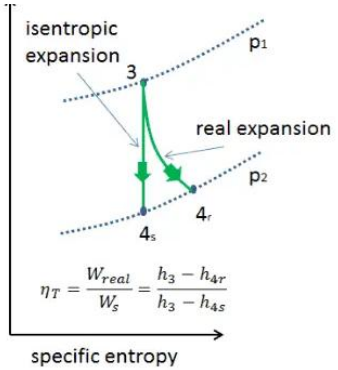



```
def Compression(gas_in: ct.Quantity, gas_out: ct.Quantity, PR, Eta, Polytropic_Eta = 0):
    if Polytropic_Eta == 1:···
    else: # isentropic efficiency
        Sin = gas_in.s
        Pout = gas_in.P*PR
        gas_out.SP = Sin, Pout # get gas_out at constant s and higher P
        Hisout = gas_out.enthalpy_mass # isentropic exit specific enthalpy
        Hout = gas_in.enthalpy_mass + (Hisout - gas_in.enthalpy_mass) / Eta
        gas_out.HP = Hout, Pout
    PW = gas_out.H - gas_out.mass * gas_in.phase.enthalpy_mass
    return PW
```

Turbine expansion code

Component model implementation

- Generic function used for calculating turbine end conditions
  - Use Cantera
  - Eta isentropic or polytropic




```
def TurbineExpansion(gas_in: ct.Quantity, gas_out: ct.Quantity, PR, Eta, Wexp, Eta_Polytropic = 0):
    Pout = gas_in.P / float(np.asarray(PR).squeeze())
    if Eta_Polytropic: ...
    else:
        gas_out.SP = gas_in.entropy_mass, Pout
        final_enthalpy_is = gas_out.enthalpy_mass
        # eta_is = (initial_enthalpy - final_enthalpy) / (initial_enthalpy - final_enthalpy_is)
        final_enthalpy = gas_in.enthalpy_mass - (gas_in.enthalpy_mass - final_enthalpy_is) * Eta
        gas_out.HP = final_enthalpy, Pout
    if Wexp == None: ...
    else:
        PW = Wexp * (gas_in.enthalpy_mass - gas_out.enthalpy_mass)
    return PW
```

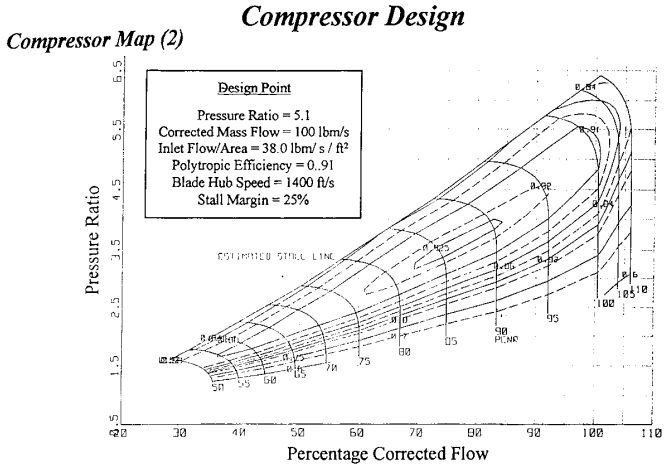
Component characteristics or “maps”

- Tabulated relations of performance parameters
- Applicable to various component models
- Examples
  - Simple: non-linear pressure loss relationship
  - Complex: turbo machinery maps:
    - fans, compressors, turbines
- Turbomachinery
  - Parameters ‘corrected’ or ‘reduced’ to ISA
  - Nc, Wc, PR, Eta, (Re)
- Scaled vs. unscaled maps

Turbomachinery maps

Component model implementation

- GSP and Gasturb map format
- Relations among
  - Nc, Wc, PR, Eta, (Re)



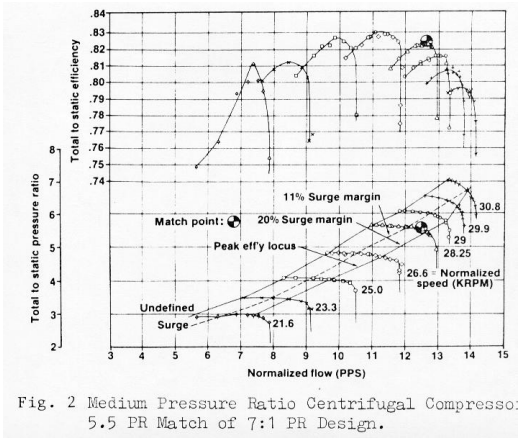
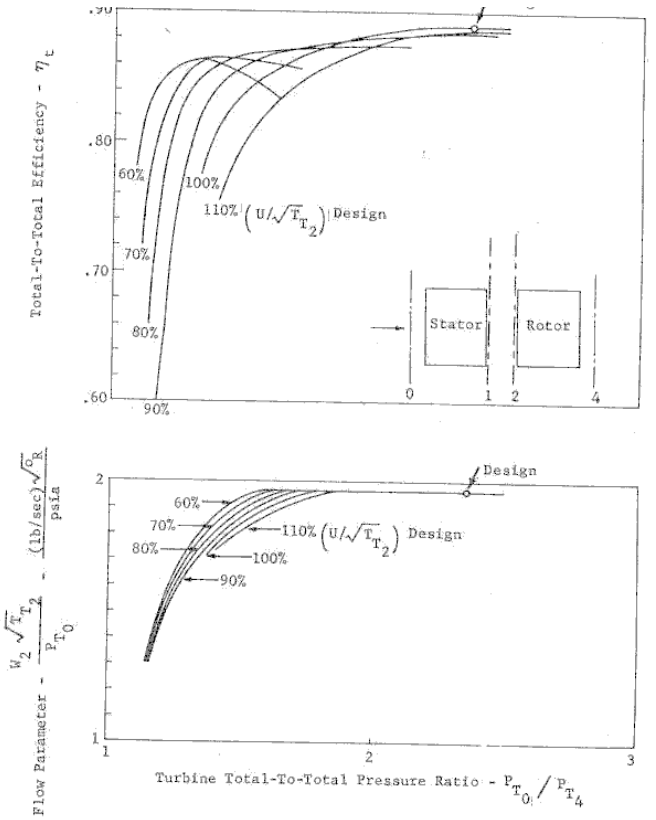


Fig. 2 Medium Pressure Ratio Centrifugal Compresso
5.5 PR Match of 7:1 PR Design.

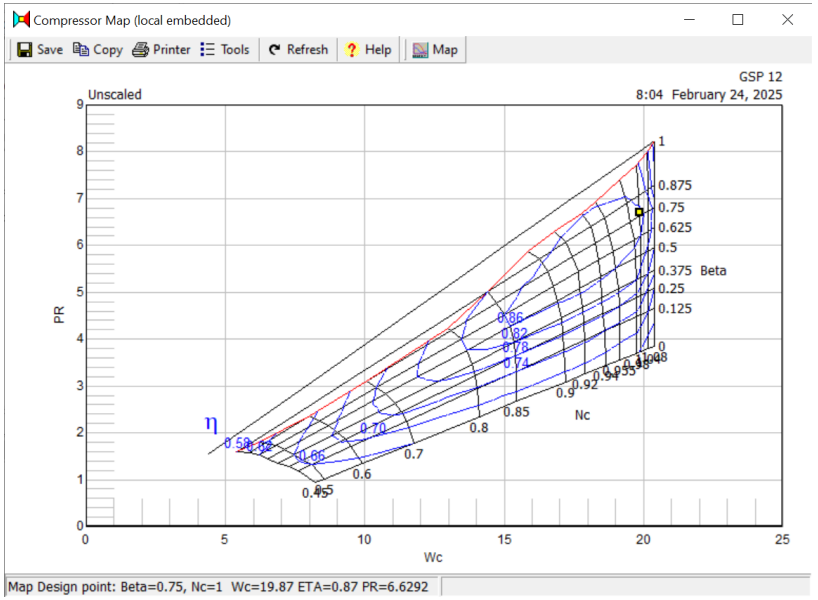


Compressor maps **Turbine map**

Maps in component models

Component model implementation

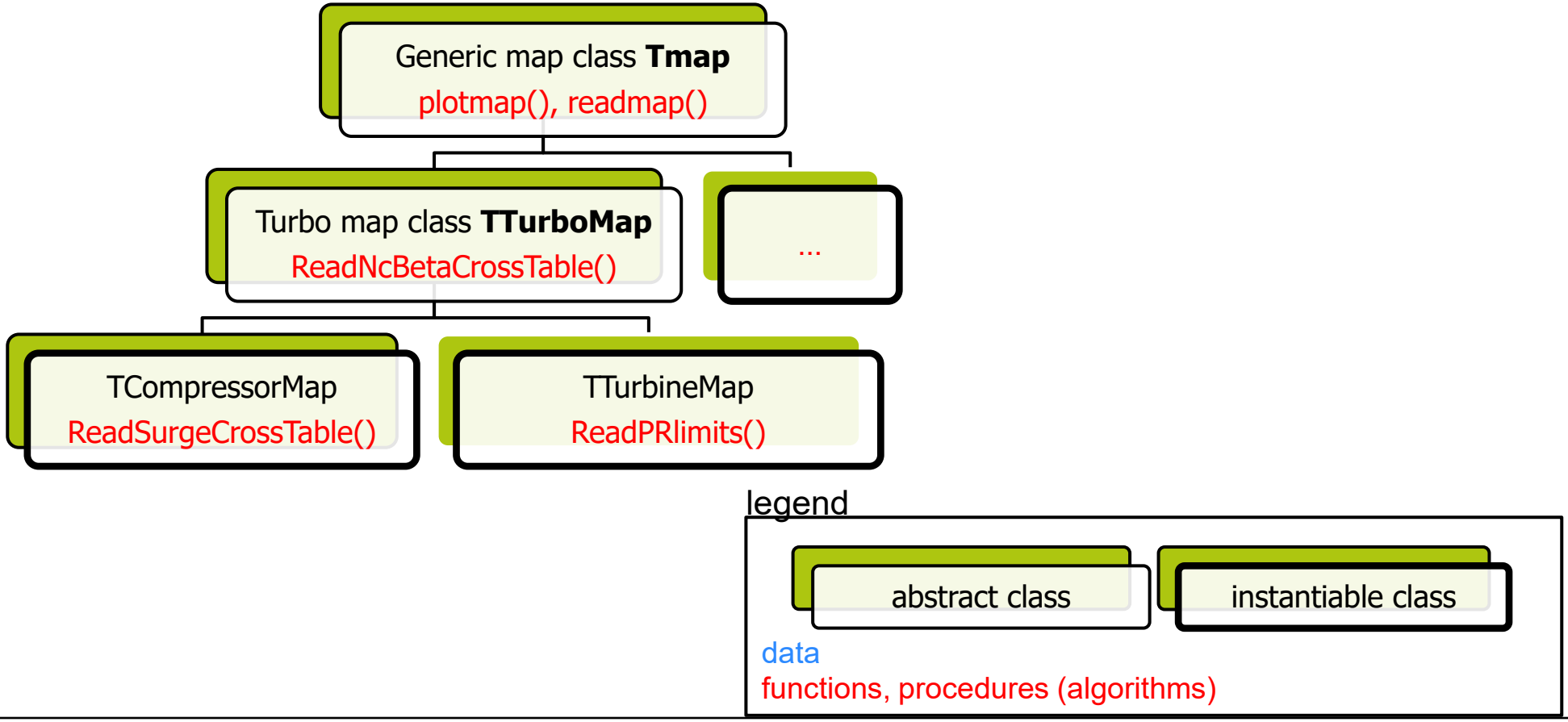
- 1 or more map parameters define component operating point
- For turbomachinery
  - 2 parameters define operating point
    - Nc
    - Beta (auxiliary parameter)



- With values for Nc and Beta, PR, Wc and Eta can be obtained from lookup tables
  - SciPy *RegularGridInterpolator* cubic interpolation

Turbomachinery map classes

Component model implementations



Plotting turbomachinery maps

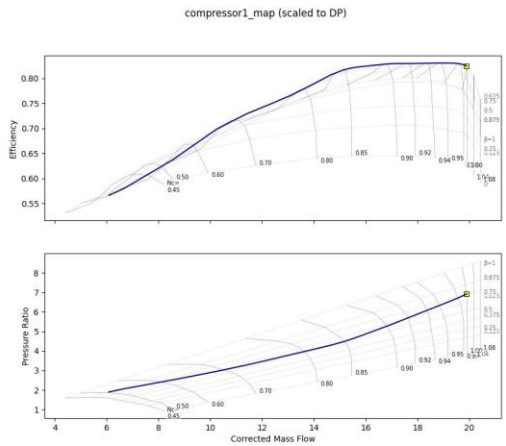
Component model implementation

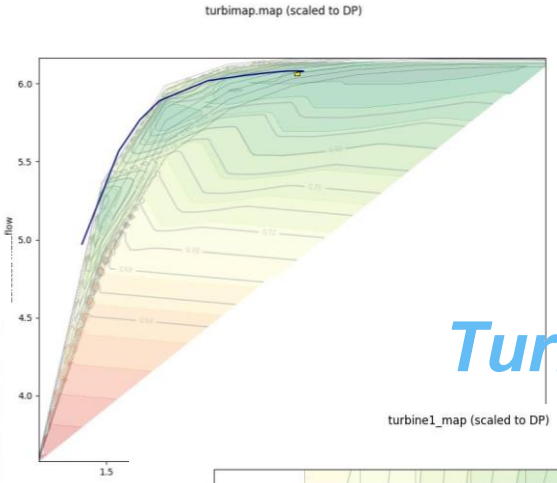
- Consider the following data when plotting
  - Pressure ratio, Mass flow, Spool speed
  - Efficiency contours
  - Limits
    - Surge line for compressor
- Design point / CRP (cycle reference point)
- Map scaling to CRP
  - For OD operating lines (transient or steady state)
- Use standard plot libraries! matplotlib

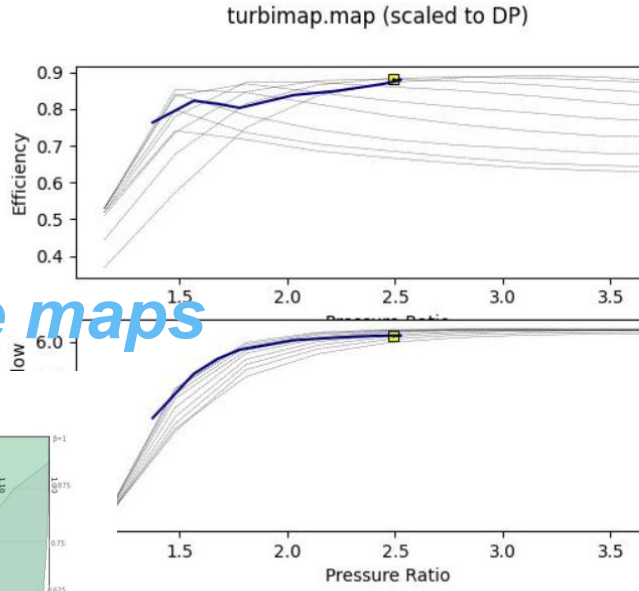
Turbomachinery map plot examples

Component model implementations

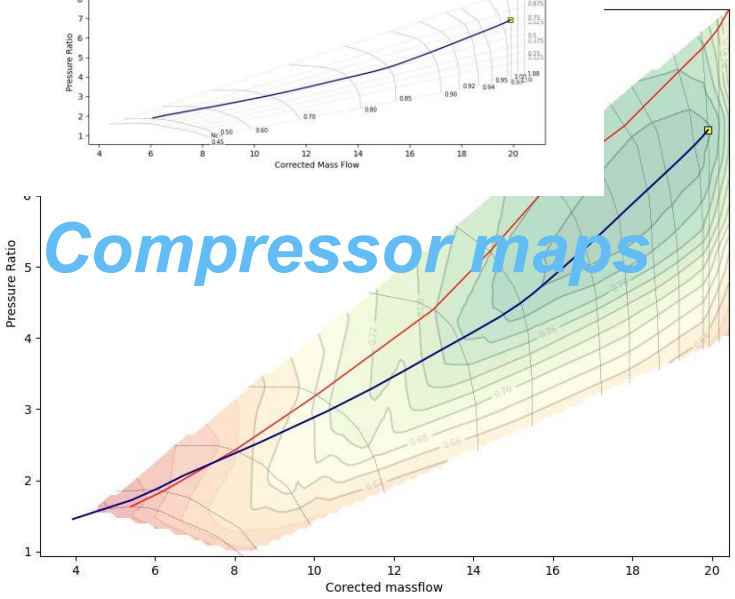
- Few lines of code!
- OD operating curves in scaled maps



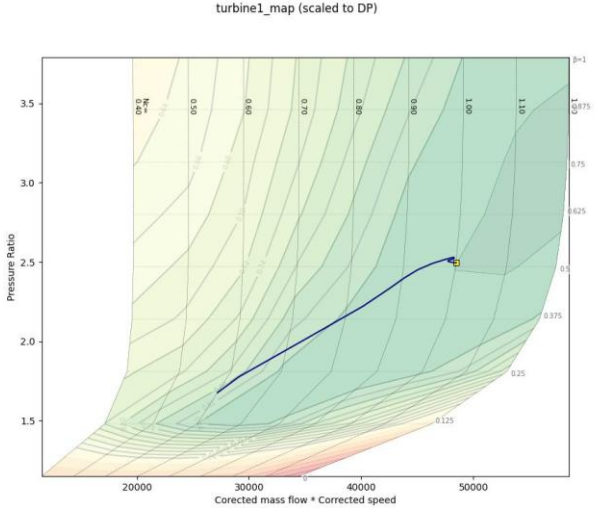




Turbine maps



Compressor maps



Python implementation - output

Component model implementation

- Generate global output table *pandas*
  - Code output per component
  - System level output (PW, FN, Wf, W2 etc.)
  - Export to .csv
  - Make graphs in Excel (or separate Python plot program)
- Component maps *matplotlib*
  - Graphical output
  - Compressors, fans, turbines, …
  - Draw operating lines in scaled maps

Demonstration



- Running single spool turbojet model
  - Environment Microsoft VScode
  - Design + Off-design to IDLE at ISA SL
- 2 spool Turbofan
  - Design T/O SL ISA
  - Off-design at Ma 0.8 / 11,000 m

Using the GSPy code (version 2.0)

Software implementation

- https://github.com/wvisser1958/GSPy *
- Limited set of component model classes/demo models
- With current open-source code you can
  - Run Design and off-design simulations of gas turbines
  - Define other engine configurations
    - Turbofan, turboshaft etc.
  - Other fuels
    - Easily adapt fuel in combustor component
- This code is the groundwork for you to continue developing your own specific models

* *The code is provided under the Apache open-source license*

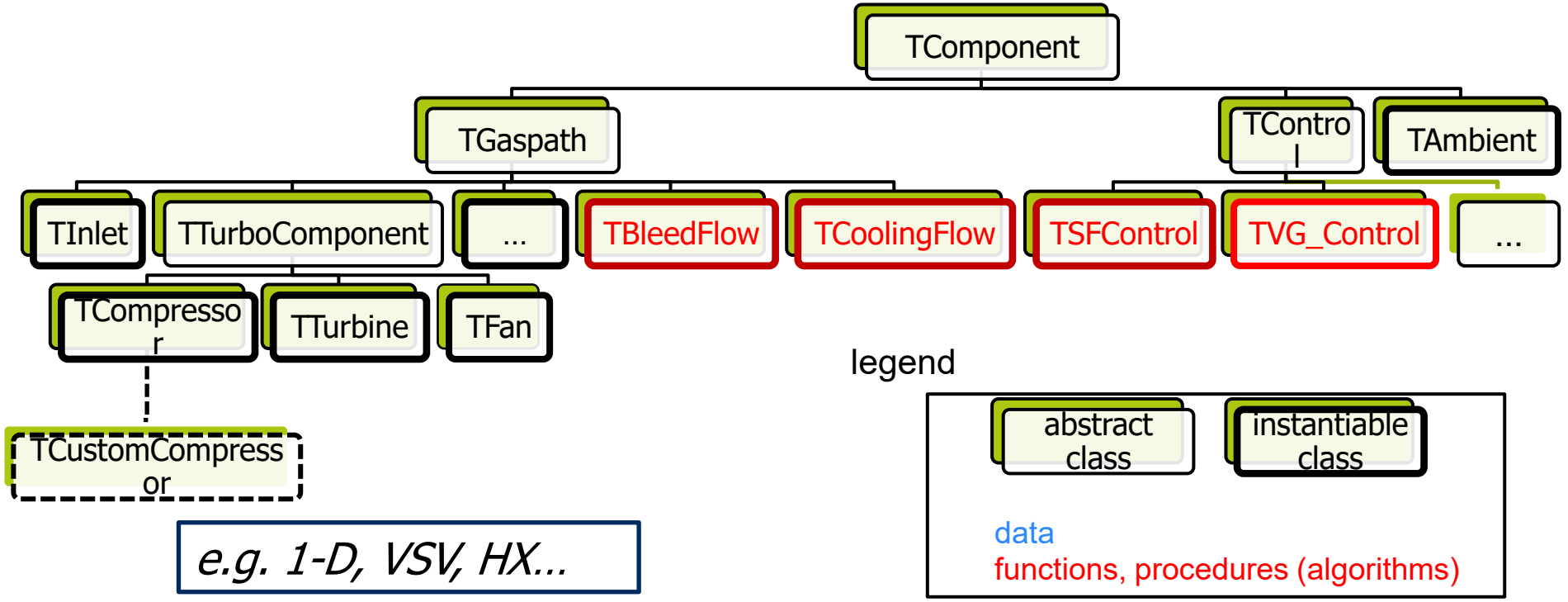
New 2026 additions

Software implementation

- Version 2.0
- Secondary airflows
  - Compressor bleed
  - Turbine cooling
- Design point equations/iteration
- Polytropic efficiency
- Separate exhaust nozzle and diffuser component models
- Adaptive modelling / test analysis
- Variable geometry (multi-map)

New component model classes (version 2.0)

Object Orientation



Compressor bleed flows

- Create list of TBleedFlow objects
- Pass list to TCompressor object

```
# compressor bleeds
compressor1bleeds = [TBleedFlow('IPbleed', None, None,
                                23, 24,     # station in, station out
                                1,          # bleed number
                                0.05,       # fraction of compressor inlet flow
                                0.5),       # dP factor: bleed extraction pressure fraction: Pbleed = Pin + dP * dPtotal
                                            # this is an improvement over the GSP method where it was proportional to dH
                                            # dPfraction can be directly determined from the bleed pressure level requirement
                    TBleedFlow('HPbleed', None, None,
                                31, 32,
                                2,
                                0.05,
                                1.0)]
# None = No bleeds
# compressor1 = TCompressor('compressor1','compmap.map' , None, 2, 3, '_gg', 4780, 0.915, 1, 0.8   , 20, 'GG', None)
compressor1 = TCompressor('compressor1',map_path / 'compmap.map' , None, 2, 3, '_gg', 4780, 0.915, 1, 0.8 , 20, 'GG', compressor1bleeds)
```

Turbine cooling flows

- Create list of TCoolingFlow objects
  - Cooling flows tap air from a bleed flow objects
- Pass list to TTurbine object

```
# GGT cooling flows
GGTcoolingflows = [TCoolingFlow('GGTcooling1', None, None,
                                32, 35, # stations in out
                                1,      # cooling flow nr
                                2,      # from bleed flow nr
                                0.5,    # fraction of bleed flow taken
                                1.0,    # dP fraction (extend to which expansion contributes to turbine power)
                                1.0,    # Wc fraction (fraction of cooling flow Wc taking up turbine flow cross area
                                0),     # pumpin radius (only > 0 for rotor blades)
                   TCoolingFlow('GGTcooling2', None, None,
                                32, 36,
                                2,
                                2,
                                0.5,
                                0.5,
                                0.5,
                                0.16)]

turbine_gg      =  TTurbine(   'GGT'   ,map_path / 'turbimap.map', None, 4, 45, '_gg', 4780, 0.8 , 1, 0.50943, 0.99, 'GG', GGTcoolingflows)
```

Turbomachinery variable geometry

- Single map
  - Set map scaling parameter values (e.g.: Wc values for a speed line)
  - E.g. using Tcontrol child class
- Multi-map
  - Use TVG_Control
  - More accurate
  - Need multiple maps (for varying VG values)

```
VG_control = TVG_Control('vg_control1', None, 6200, 0)
compressor1 = TCompressor('CompIGV_0',
                          {"design_angle": 0,
                           "maps": {-5: map_path / "map_-10.map",
                                    0: map_path / "map_0.map",
                                   10: map_path / "map_10.map",
                                   20: map_path / "map_20.map",
                                   30: map_path / "map_30.map"}},
                         VG_control, 2, 3, '_gg', 6200, 0.90, 1, 0.78   , 20, 'GG', compressor1bleeds)
```

Future

Software implementation

- 3.0 release
  - Water liquid/vapor phase change
    - *TFlowState* class shell around Cantera
    - Water injection
  - Thermal effects / Thermal network model
    - W. P. J. Visser and I. D. Dountchev, "Modeling Thermal Effects on Performance of small Gas Turbines", GT2015-42744

available as beta under version 2.1

- SAE ARP4868 Application Programmers Interface (API)
- Transient simulation
  - Extending the conservation equations with time derivatives

Extending the baseline code - 1

Software implementation

- Adding component model classes
  - WaterInjector *(added in 3.0)*
  - Mixer
  - Recuperator / heat exchanger

- Additional *Hybrid* propulsion system elements
  - Generators / Loads *(added in 3.0)*
  - Electric motors *(added in 3.0)*
  - Propulsors
  - Batteries,
  - Fuel cells

Extending the baseline code - 2

Software implementation

- Extend user interface
  - Extend output Graphs, tables, export
  - Add GUI
- Coupling with other models
  - Use SAE standard (ARP4868 API)
  - Running *GSPy* code via API
  - *GSPy* controlling execution of other models
- Fidelity
  - 1-D component models
    - Turbomachinery
    - Combustors

Conclusion

- GSPy open-source code available from
  - https://github.com/wvisser1958/GSPy
- The GSPy code is rapidly evolving
  - Users / developers in academia, government and industry
- Contact authors for questions
  - Dr. Wilfried Visser w.p.j.Visser@tudelft.nl
  - Oscar Kogenhop MSc oscar.kogenhop@epcor.nl

Questions?

